MGT 205 in class - Star Wars

Keman Xiang

Basic Functionality of R and Variables

Q1. Data Types

a)

R in its most basic form can be used as a calculator. Write some coded math expressions using at least three different operators (including at least one trigonometric function).

Note: R follows the standard order of operations, but this can be controlled by wrapping any expression in ().

```
2/1
```

```
## [1] 2
```

```
5+2
```

```
## [1] 7
```

```
3*6
```

```
## [1] 18
```

b)

Name and give an example of the five base data types in R:

```
# Character
'some text'
```

```
## [1] "some text"
```

```
# Numeric
123.456
```

```
## [1] 123.456
```

```
# Integer
1234L
```

```
## [1] 1234
```

```
# Logical
TRUE
```

```
## [1] TRUE
```

```
# Complex
2 + 3i
```

```
## [1] 2+3i
```

c)

Construct a vector in R.

A vector is a data structure in R that contains multiple entries of the same type.

```
c('hello','mgt','205')
```

```
## [1] "hello" "mgt"   "205"
```

Q2. Variables

Variables are a reference to data in R and can be assigned any valid data structure. Variables can be used anywhere the data that they are referencing could be and serve as the building blocks for working with data in R as they can be composed.

a)

Create numeric variables x and y independently, then add x and y to a new variable z.

```
x <- 1
y <- 5
z <- x+y
z
```

```
## [1] 6
```

Q3. Functions

- Built in functions R's built in functions known as 'base R' form an incredibly powerful statistical package that can be used in a range of ways to extract information from data. Here we cover the basic usage of very common R functions.

a)

i)

Create a sequence from 1 up to 10

```
seq(1,10)
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10
```

ii)

What is the sum of all numbers from 1 to 12345?

```
sum(1:12345)
```

```
## [1] 76205685
```

b)

Generate 100 normally distributed random numbers

```
rnorm(100)
```

```
##   [1] -0.5419540144 -0.8861002659 -1.6075581144  1.4947543215 -0.2407255558
##   [6]  0.9777472678 -0.1269955625  0.2505352413 -1.8968402877  0.9520650863
##  [11]  1.7947237280 -0.3307333083 -1.0866388939 -0.5544188750 -2.1284124570
##  [16] -0.4385094321 -0.9322883962 -1.1332040939  1.2092108096 -1.4183042722
##  [21] -0.0294024169 -1.1494623923 -1.2763837174  0.5066701331  1.3759006670
##  [26] -1.2119569291  0.5891552402  0.7023425464 -0.0892655884 -0.2875422308
##  [31] -0.3395697802 -0.4969991426 -0.7055151445  0.5186434958  0.0599389881
##  [36] -0.2903117819 -1.2656559462 -1.2403394509 -0.2075141152  0.6986053952
##  [41] -1.7959036817  1.4056661011  0.0452558528 -0.5444423712 -0.4979762859
##  [46] -0.0157950807 -1.6811657468 -0.0807584915 -1.3083359622  0.4084021298
##  [51] -1.7989756748 -1.3388849167  0.6482296173 -0.3215193528 -0.4470942100
##  [56] -1.3762809187 -1.0771670661  0.2900688789 -0.3581935384  0.7535531653
##  [61] -0.3109121890  0.4852198599 -0.2037785705  0.0003597108  0.0836047282
##  [66] -0.3511958826  0.5140518140 -0.4476319090  2.2151640763 -0.0960543665
##  [71] -0.6941052531 -0.1344018161  0.5781819140  1.5715339761  0.9719121817
##  [76]  0.7436565622 -0.2810959288 -0.7016182364 -1.7125118427  0.2982242475
##  [81] -0.3318255013  0.0873833152 -0.3597291139  0.2454916916 -0.3817840704
##  [86] -0.0298915601 -0.6373184738 -0.6854586188  1.5098773963 -0.5822545877
##  [91] -0.5692043349  1.9807791284 -1.5900479704  0.5919805523  0.5552232219
##  [96]  0.0487541649  0.0313167917  1.5778980111 -0.8485452358 -2.7101660229
```

c)

i)

Sample 30 random whole numbers between 1 and 500

```
sample(seq(1,500),30)
```

```
##  [1]  52 399 260 135 172 196 335 257  94 460  69 205 118 467 209 124 465 423 383
## [20]  90 427 169  19 300 479 408 319 318 463 331
```

ii)

Choose and show 10 randomly generated normally distributed numbers

```
sample(rnorm(10))
```

```
##  [1]  1.3900334  0.4712640 -1.5934401  1.0899331  0.2211505  0.5083182
##  [7]  0.9209278  1.1365627  0.2089542 -0.2071011
```

d)

Simulate flipping a coin 100 times with with 1s and 0s standing for heads and tails respectively. Count the number of heads.

```
sum(sample(c(0,1),size=100,replace=TRUE))
```

```
## [1] 51
```

R Packages and functions

'R packages' are collections of useful functions created by academics and industry professionals that have been published openly for use by anyone. One of the most widely used collection of packages is the tidyverse, which provides a consistent way of working with data from the beginning of an analysis to the end.

User defined functions

Functions can be defined as needed to simplify operations or reduce repetition in code. The function() keyword is used to do this. 'Arguments' are passed to functions when they are called with parentheses and are separated by commas. These arguments can be accessed inside the function.

e)

Define a function called add_one that adds 1 to its input.

```
add_one <- function(num){
 num + 1
}
```

f)

Define a function that takes 2 arguments and randomly returns one.

```
pick_one <- function(one, two){
 vec <- c(one, two)
 sample(vec, size = 1)
}
```

Reading Data and Working with Data Frames

From here on out we will be using the tidyverse. The tidyverse provides a consistent way to work with data, and extends the functionality of base R. In particular, we will be using dplyr for data processing, along with ggplot2 for visualization and readr for data reading.

Q1. Reading Data

Run the code block below.

```
library(tidyverse)
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✔ dplyr     1.1.4     ✔ readr     2.1.4
## ✔ forcats   1.0.0     ✔ stringr   1.5.1
## ✔ ggplot2   3.4.4     ✔ tibble    3.2.1
## ✔ lubridate 1.9.3     ✔ tidyr     1.3.0
## ✔ purrr     1.0.2
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

a)

Load in the starwars data.

```
starwars = read_csv("starwars.csv")
```

```
## Rows: 87 Columns: 11
## ── Column specification ────────────────────────────────────────────────────────
## Delimiter: ","
## chr (8): name, hair_color, skin_color, eye_color, sex, gender, homeworld, sp...
## dbl (3): height, mass, birth_year
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

b)

List the first 6 rows of starwars.

```
head(starwars)
```

```
## # A tibble: 6 × 11
##   name      height  mass hair_color skin_color eye_color birth_year sex   gender
##   <chr>      <dbl> <dbl> <chr>      <chr>      <chr>          <dbl> <chr> <chr>
## 1 Luke Sky…    172    77 blond      fair       blue            19   male  mascu…
## 2 C-3PO        167    75 <NA>       gold       yellow         112   none  mascu…
## 3 R2-D2         96    32 <NA>       white, bl… red             33   none  mascu…
## 4 Darth Va…    202   136 none       white      yellow          41.9 male  mascu…
## 5 Leia Org…    150    49 brown      light      brown           19   fema… femin…
## 6 Owen Lars    178   120 brown, gr… light      blue            52   male  mascu…
## # ℹ 2 more variables: homeworld <chr>, species <chr>
```

c)

Get the number of rows and number of columns in starwars.

```
nrow(starwars)
```

```
## [1] 87
```

```
ncol(starwars)
```

```
## [1] 11
```

```
dim(starwars)
```

```
## [1] 87 11
```

Q2. Data Pipelines (%>%)

Often multiple processing steps are required for data frames, but the intermediate results are often not that important. There are a number of ways to apply functions in succession, but the preferred standard is to use the forward pipe operator %>%. This operator takes whatever expression is on its left hand side and uses this expression as the first argument of the function on its right hand side. i.e. x %>% f %>% g %>% h is equivalent to h(g(f(x)))

https://r4ds.had.co.nz/pipes.html https://magrittr.tidyverse.org

a)

Take the head of starwars and find how many rows are in it.

```
starwars %>%
 head() %>%
 nrow()
```

```
## [1] 6
```

Q3. Working with Data Frames

a)

i)

How many males are in the data set?

```
starwars %>%
  filter(sex == 'male') %>%
  nrow()
```

```
## [1] 60
```

ii)

How many humans with blue eyes are in the data set?

```
starwars %>%
  filter(species == 'Human', eye_color == 'blue') %>%
  nrow()
```

```
## [1] 12
```

b)

List the tallest 3 humans, showing only name and height columns.

```
starwars %>%
  filter(species == 'Human') %>%
  arrange(desc(height)) %>%
  head(3) %>%
  select(name, height)
```

```
## # A tibble: 3 × 2
##   name          height
##   <chr>          <dbl>
## 1 Darth Vader      202
## 2 Qui-Gon Jinn     193
## 3 Dooku            193
```

c)

Show the mean height and mass of humans.

i)

```
starwars %>%
  filter(species == 'Human') %>%
  summarise(
    mean_height = mean(height, na.rm = TRUE),
    mean_mass = mean(mass, na.rm = TRUE)
  )
```

```
## # A tibble: 1 × 2
##   mean_height mean_mass
##         <dbl>     <dbl>
## 1        177.      82.8
```

ii)

Show the mean height and mass of humans grouped by sex.

```
starwars %>%
  filter(species == 'Human') %>%
  group_by(sex) %>%
  summarise(
    mean_height = mean(height, na.rm = TRUE),
    mean_mass = mean(mass, na.rm = TRUE)
  )
```

```
## # A tibble: 2 × 3
##   sex    mean_height mean_mass
##   <chr>        <dbl>     <dbl>
## 1 female        160.      56.3
## 2 male          182.      87.0
```

d)

Get the 2 shortest masculine and feminine characters, displaying only name, height and gender columns (you must remove missing values for gender).

```
starwars %>%
  filter(gender == 'masculine') %>%
  arrange(height) %>%
  head(2) %>%
  select(name, height, gender)
```

```
## # A tibble: 2 × 3
##   name          height gender
##   <chr>          <dbl> <chr>
## 1 Yoda              66 masculine
## 2 Ratts Tyerell     79 masculine
```

```
starwars %>%
  filter(gender == 'feminine') %>%
  arrange(height) %>%
  head(2) %>%
  select(name, height, gender)
```

```
## # A tibble: 2 × 3
##   name        height gender
##   <chr>        <dbl> <chr>
## 1 R4-P17          96 feminine
## 2 Leia Organa    150 feminine
```

Data Visualization

Base R has its own plotting utilities, but they are rarely for creating presentable visualizations as they are difficult to configure and do not have consistent behavior for different plot types. We will be using the ggplot2 package, which is the industry standard for data visualization as it uses a consistent 'grammar' to construct all types of visualizations, produces high quality outputs by default and is easy to configure.

We will also be using the palmerpenguins educational data package for the data underlying our plots.

Run the code block below.

```
#install.packages("palmerpenguins")
library(palmerpenguins)
```

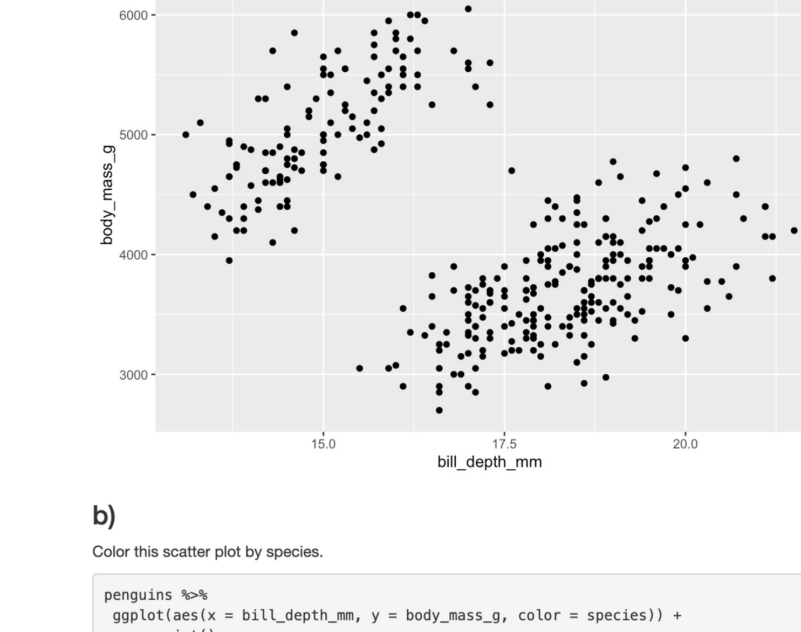
Q1)

a)

Create a scatter plot with bill depth on the x axis and body mass on the y axis.

```
penguins %>%
 ggplot(aes(x = bill_depth_mm, y = body_mass_g)) +
 geom_point()
```

```
## Warning: Removed 2 rows containing missing values (`geom_point()`).
```

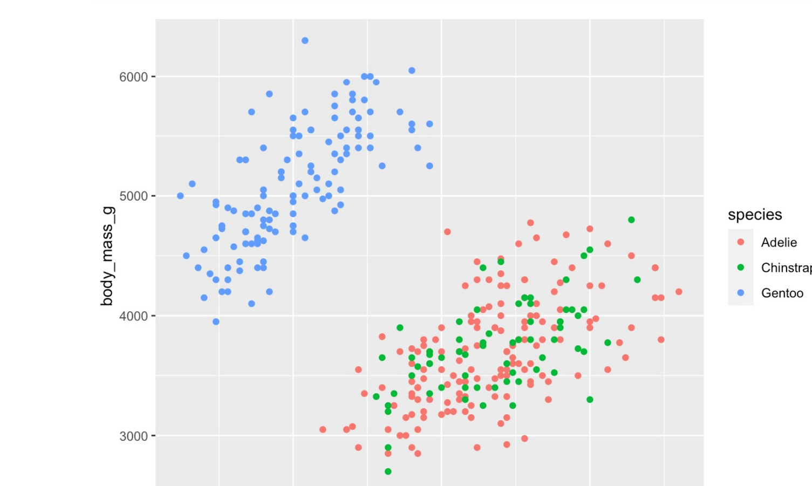


b)

Color this scatter plot by species.

```
penguins %>%
 ggplot(aes(x = bill_depth_mm, y = body_mass_g, color = species)) +
 geom_point()
```

```
## Warning: Removed 2 rows containing missing values (`geom_point()`).
```

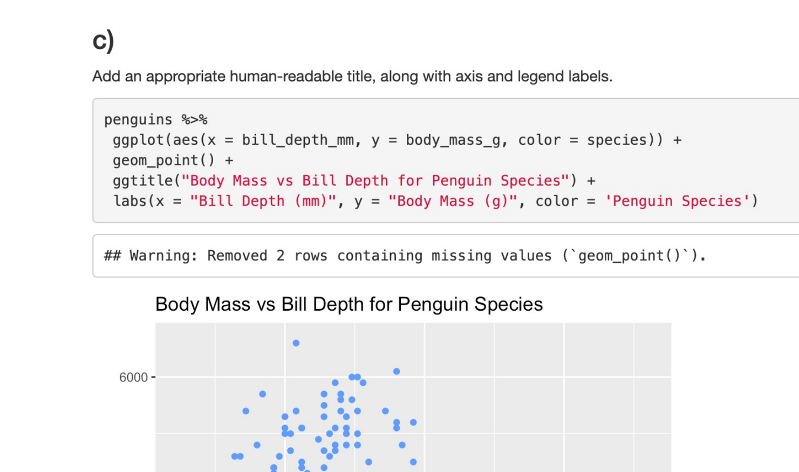


c)

Add an appropriate human-readable title, along with axis and legend labels.

```
penguins %>%
 ggplot(aes(x = bill_depth_mm, y = body_mass_g, color = species)) +
 geom_point() +
 ggtitle("Body Mass vs Bill Depth for Penguin Species") +
 labs(x = "Bill Depth (mm)", y = "Body Mass (g)", color = 'Penguin Species')
```

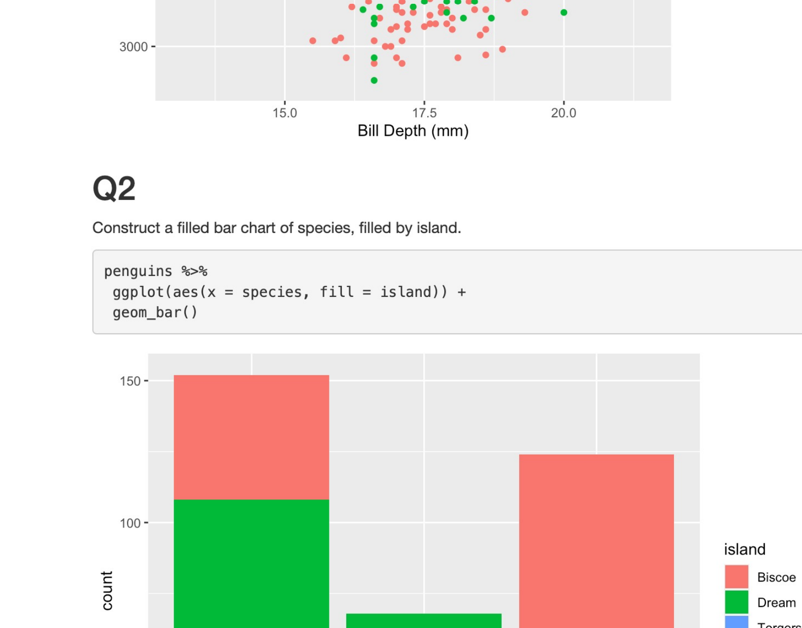
```
## Warning: Removed 2 rows containing missing values (`geom_point()`).
```



Q2

Construct a filled bar chart of species, filled by island.

```
penguins %>%
 ggplot(aes(x = species, fill = island)) +
 geom_bar()
```



Q3

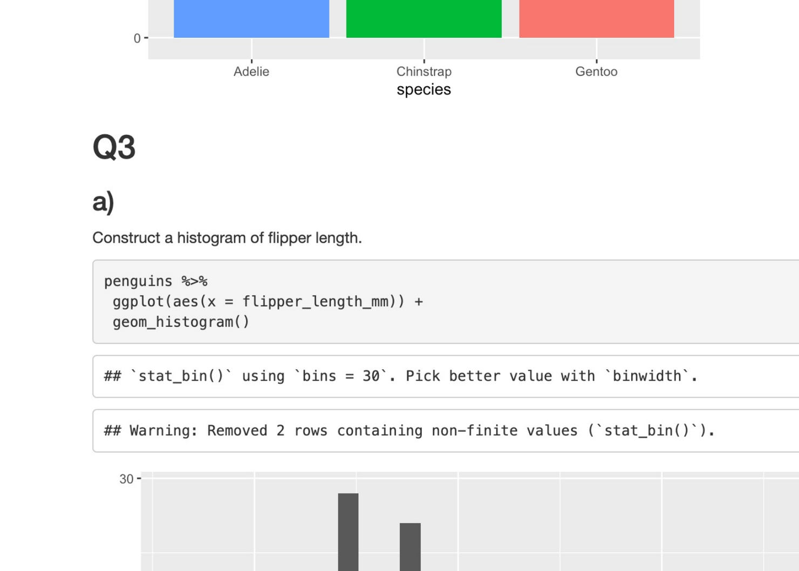
a)

Construct a histogram of flipper length.

```
penguins %>%
 ggplot(aes(x = flipper_length_mm)) +
 geom_histogram()
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

```
## Warning: Removed 2 rows containing non-finite values (`stat_bin()`).
```



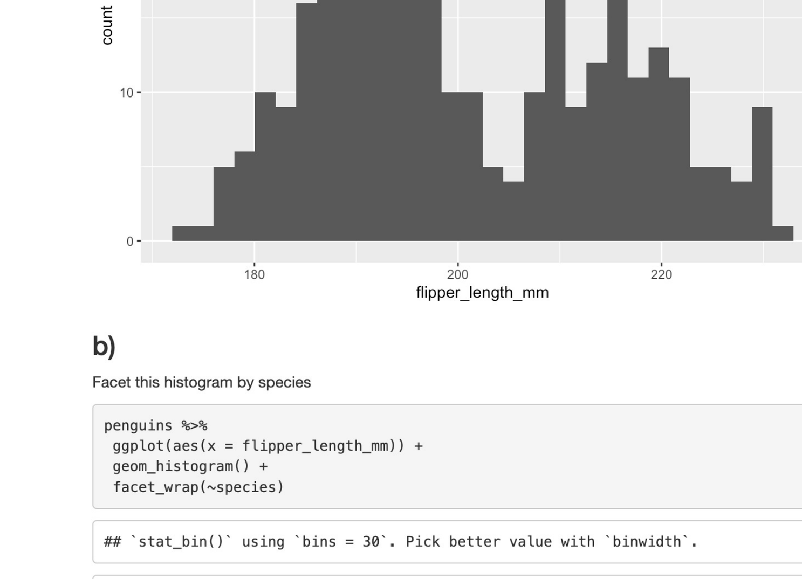
b)

Facet this histogram by species

```
penguins %>%
 ggplot(aes(x = flipper_length_mm)) +
 geom_histogram() +
 facet_wrap(~species)
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

```
## Warning: Removed 2 rows containing non-finite values (`stat_bin()`).
```



c)

Compare the flipper length distributions between different species using density geom with an alpha.

```
penguins %>%
 ggplot(aes(x = flipper_length_mm, fill = species)) +
 geom_density(alpha = 0.2)
```

```
## Warning: Removed 2 rows containing non-finite values (`stat_density()`).
```